Volume I: Core Foundations

ReactJS Solutions Manual

Complete Implementation Blueprint for Labs 1-5

Included Modules:

- **Lab 1:** Environment Provisioning & Scaffolding (`myfirstreact`)
- **Lab 2:** Hierarchical Class Component Nesting (`StudentApp`)
- **Lab 3:** Functional Component Architecture & Prop Configurations (`scorecalculatorapp`)
- **Lab 4:** Async Data Provisioning & Lifecycle Hook Handling (`blogapp`)

- **Lab 5:** Local CSS Modules Scoping & Color Mutation Layers (`cohortstracker`)

Lab 1: Environment Setup & Your First React Application

OBJECTIVES MET

- Differentiated Single-Page Applications (SPA) from Multi-Page Applications (MPA).
- Utilized automated scaffolding frameworks (`create-react-app`).

Step 1: Application Generation

Terminal Context

```
npx create-react-app myfirstreact
cd myfirstreact
npm start
```

Step 2: Core Customization

src/App.js

```
import React from 'react';

function App() {
  return (
    <div>
      <h1>welcome to the first session of React</h1>
    </div>
  );
}

export default App;
```

Lab 2: Class Components & Multi-Component Synthesis

OBJECTIVES MET

- Constructed stateful class-based architecture blocks.
- Synchronized multi-component rendering pipelines inside a layout manager wrapper.

Step 1: Creating Child Components (src/Components/)

src/Components/Home.js

```
import React, { Component } from 'react';
class Home extends Component {
  render() {
    return <p>Welcome to the Home page of Student Management Portal</p>;
  }
}
export default Home;
```

src/Components/About.js

```
import React, { Component } from 'react';
class About extends Component {
  render() {
    return <p>Welcome to the About page of the Student Management Portal</p>;
  }
}
export default About;
```

src/Components/Contact.js

```
import React, { Component } from 'react';
class Contact extends Component {
  render() {
    return <p>Welcome to the Contact page of the Student Management Portal</p>;
  }
}
export default Contact;
```

Step 2: Main Application Pipeline Orchestration

src/App.js

```
import React, { Component } from 'react';
import Home from './Components/Home';
import About from './Components/About';
import Contact from './Components/Contact';

class App extends Component {
  render() {
    return (
      <div>
        <Home />
        <About />
        <Contact />
      </div>
    );
  }
}
export default App;
```

Lab 3: Functional Components, Dynamic Props & Custom CSS

OBJECTIVES MET

- Implemented unidirectional props pipeline structures inside stateless structures.
- Designed isolated custom structural sheets for metrics calculator blocks.

Step 1: Custom Style Configuration Sheets

src/Stylesheets/mystyle.css

```
.score-container {
    border: 1px solid #cbd5e1;
    padding: 15px;
    margin: 10px 0;
    background-color: #f8fafc;
}
```

Step 2: Component Engineering

src/Components/CalculateScore.js

```
import React from 'react';
import '../Stylesheets/mystyle.css';

function CalculateScore({ Name, School, Total, goal }) {
  const averageScore = (Total / goal).toFixed(2);
  return (
    <div className="score-container">
      <h2>Student Score Summary</h2>
      <p><strong>Name:</strong> {Name}</p>
      <p><strong>School:</strong> {School}</p>
      <p><strong>Total Score:</strong> {Total}</p>
      <p><strong>Goal:</strong> {goal}</p>
      <p><strong>Calculated Average:</strong> {averageScore}</p>
    </div>
  );
}
export default CalculateScore;
```

Step 3: Execution Context

src/App.js

```
import React from 'react';
import CalculateScore from './Components/CalculateScore';

function App() {
  return (
    <div>
      <CalculateScore
        Name="John Doe"
        School="Cognizant Academy"
        Total={450goal={5/>
    </div>
  );
}
export default App;
```

Lab 4: Component Lifecycle Hooks & Async Fetch Operations

OBJECTIVES MET

- Implemented asynchronous pipeline triggers within `componentDidMount()`.
- Handled exceptions using custom component-level `componentDidCatch()` parameters.

Step 1: Domain Entity Class Mapping

src/Post.js

```
class Post {
  constructor(id, title, body) {
    this.id = id;
    this.title = title;
    this.body = body;
  }
}
export default Post;
```

Step 2: Network-Aware Lifecycle Sub-System Layer

src/Posts.js

```
import React, { Component } from 'react';

class Posts extends Component {
  constructor(props) {
    super(props);
    this.state = { posts: [] };
  }

  loadPosts() {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(response => response.json())
      .then(data => {
        this.setState({ posts: data.slice(0, 5) });
      });
  }

  componentDidMount() {
    this.loadPosts();
  }

  componentDidCatch(error, info) {
    alert('Error intercepted inside lifecycle loop: ' + error);
  }

  render() {
    return (
      <div>
        <h2>Fetched Post Feed Data</h2>
        {this.state.posts.map(post => (
          <div key={post.idstyle={{ borderBottom: '1px solid #ccc', padding: '10px' >
            <h3>{post.title}</h3>
            <p>{post.body}</p>
          </div>
        ))}
      </div>
    );
  }
}
export default Posts;
```

Step 3: Root Execution Bridge

src/App.js

```
import React from 'react';
import Posts from './Posts';

function App() {
  return (
    <div>
      <Posts />
    </div>
  );
}
export default App;
```

Lab 5: Scoped Style Isolation via CSS Modules & Status Mutation

OBJECTIVES MET

- Isolated scoping boundaries using `*.module.css` local hash naming setups.
- Applied dynamic logical mappings using status variables to set typography colors.

Step 1: Local CSS Module Rule Block Declarations

src/CohortDetails.module.css

```
.box {
    width: 300px;
    display: inline-block;
    margin: 10px;
    padding: 10px 20px;
    border: 1px solid #000000;
    border-radius: 10px;
    vertical-align: top;
    background-color: #ffffff;
}

dt {
    font-weight: 500;
}
```

Step 2: Component Engineering with State Formatting Controls

src/CohortDetails.js

```
import React from 'react';
import styles from './CohortDetails.module.css';

function CohortDetails({ cohort }) {
  const isOngoing = cohort.status.toLowerCase() === 'ongoing';
  const titleStyle = { color: isOngoing ? 'green' : 'blue' };

  return (
    <div className={styles.box>
      <h3 style={titleStyle>{cohort.code} - {cohort.title}</h3>
      <dl>
        <dt>Started On</dt>
        <dd>{cohort.startDate}</dd>
        <dt>Current Status</dt>
        <dd>{cohort.status}</dd>
        <dt>Coach</dt>
        <dd>{cohort.coach}</dd>
        <dt>Trainer</dt>
        <dd>{cohort.trainer}</dd>
      </dl>
    </div>
  );
}
export default CohortDetails;
```

Step 3: Root Workspace Application File

src/App.js

```
import React from 'react';
import CohortDetails from './CohortDetails';

function App() {
  const cohortData = [
    { code: 'INTADMDF10', title: '.NET FSD', startDate: '22-Feb-2022', status: 'Scheduled',
coach: 'Aathma', trainer: 'Jojo Jose' },
    { code: 'ADM21JF014', title: 'Java FSD', startDate: '10-Sep-2021', status: 'Ongoing',
coach: 'Apoorv', trainer: 'Elisa Smith' },
    { code: 'CDBJF21025', title: 'Java FSD', startDate: '24-Dec-2021', status: 'Ongoing',
coach: 'Aathma', trainer: 'John Doe' }
  ];

  return (
    <div style={{ padding: '20px' >
      <h2>Cohorts Details Dashboard</h2>
      <div>
        {cohortData.map((item, ix) => <CohortDetails key={ixcohort={item/>)}
      </div>
    </div>
  );
}
export default App;
```